String View Conversion for Function Arguments

ISO/IEC JTC1 SC22 WG21 P0994R0

ADAM David Alan Martin (adam@recursive.engineer)
                        (adam.recursive.engineer@gmail.com)
                        (adam.martin@mongodb.com)
Jason Carey   (jason.carey@mongodb.com)

Abstract

Code wishing to call a function with a `std::string_view` parameter is unable to do so presently with a type that has a user-defined conversion to `std::string` or `const char *`. This was discussed in EWG at Jacksonville during a session discussing pain points in modern C++. (P0922r0) I (ADAM Martin) proposed that this is an easy to solve problem with current C++ library design techniques, and it does not require any new features be added to the language. This paper codifies that proposal.

We propose adding a new implicit conversion constructor to `std::basic_string_view< CharT >` which facilitates user defined conversions from any type to `std::basic_string< CharT >`, `const CharT *`, and `const std::basic_string< CharT > &`. This constructor will be invoked as an implicit conversion construction of `std::basic_string_view< CharT >`, adapting to any user defined object with a user defined "classical" string conversion operator. This constructor also includes a facility to extend the lifetime of any ephemeral `std::basic_string< CharT >` object created in the process of adapting a `std::basic_string_view< CharT >`, in a manner similar to builtin lifetime extension.

Proposal

We propose to add a new constructor to the `std::basic_string_view` family which solves this problem. This constructor must be an implicit constructor which is templatized and controlled with `std::enable_if`.

This constructor uses a technique similar to the "forwarding constructor" technique used in C++17's `std::optional< T >` conversion constructors. Essentially the technique uses an implicit conversion constructor which is controlled by an `std::enable_if` gate which prevents the instantiation of the function in certain circumstances. In the case of `std::optional< T >`, there exists a ctor, `template< typename U, ... > optional< T >::optional( U &&u )`. The elided code in elipses is a `std::enable_if` expression which, as described earlier, prevents the consideration of this ctor when the specified `U` type is not implicitly convertible to `T`.

For the case of the constructor we propose adding to `std::basic_string_view< CharT >`, it should have a similar `enable_if` constructor, `template< typename CharT > template< typename U, ... > basic_string_view< CharT >::basic_string_view( U &&u )` which is considered only when the type `U` is convertible to one of the "classical" string types -- `std::string`, `const char *`, `const std::string &`.

Some commonly expected use cases are:

Example 1

```
namespace UserCode
{
    struct Case1
    {
        std::string s= "Hello";
        operator const char *() const { return s.c_str(); }
    };
}

void function( std::string_view view ) {}

void
usage()
{
    using namespace UserCode;

    // Ephemeral usage
    function( Case1() );

    // Local usage
    Case1 c1;
    function( c1 );
}
```

In this case, the object `Case1` is responsible for the storage of the string being returned, so an anonymous temporary string view object has no issue with respect to the lifetime of the string storage returned by `Case1`'s conversion operator.

Example 2

Things become a bit more complicated in the case when a `const std::basic_string< CharT > &` is returned by the conversion operator of a type, but not significantly so:

```
namespace UserCode
{
    struct Case2
    {
        std::string s= "Hello";
```

```
        operator const std::string &() const { return s; }
    };
}

void function( std::string_view view ) {}

void
usage()
{
    using namespace UserCode;

    // Ephemeral usage
    function( Case2() );

    // Local usage
    Case2 c2;
    function( c2 );
}
```

In this case, a user-defined conversion involves a type which has lifetime extension issues, but the type returned by the user-defined conversion is a reference, thus indicating that the user defined type is responsible for the lifetime of the string data storage. A `std::basic_string_view< CharT >` constructor which accepts types with user-defined `const std::string &` operators must also decline to be eligible to convert from a temporary `std::string`, as the string's storage is ephemeral.

Example 3

Care needs to be taken when implementing some kind of magical conversion operator between a user defined type and a reference-like type (pointers, string_view, etc) when there is an intermediate step through a type which owns the resources being referenced. The C++ language has some intrinsic forms of lifetime extension, but for this kind of case, there is no facility for transitive propagation of a requirement for lifetime extension.

An example which could be at risk of this issue is:

```
namespace UserCode
{
    struct Case3
    {
        operator std::string () const { return "Hello"; }
    };
}

void function( std::string_view view ) {}

void
usage()
{
    using namespace UserCode;

    // Ephemeral usage
    function( Case3() );

    // Local usage
    Case3 c3;
    function( c3 );
}
```

Issues with any implementation attempt

In the final example above, the ephemeral `std::string` object would have its lifetime expire in a "forwarding constructor" using the `template< typename T > template< typename U > std::optional< T >::optional( U &&u )` case. The forwarding constructor's scope would end, thus ending the lifetime of the ephemeral string. A naive constructor with an implementation as detailed below would cause dangling pointers to expired storage:

```
namespace std
{
    template< typename CharT >
    class basic_string_view
    {
        public:
            template< typename U >
            basic_string_view( const U &u )
                : basic_string_view( static_cast< const std::basic_string< CharT > & >( u ) ) {}
    };
}
```

The temporary `std::string` would be created in the scope of the ctor for `basic_string_view` rather than in the calling scope, and therefore its lifetime would end before the ctor returns. We require a mechanism to extend the lifetime of temporaries created in the course of making invisible multi-stage conversions. Although this mechanism *could* be a language mechanism, it is actually possible to make such lifetime-preserving constructors in C++ today:

Introducing a trick to provide some user-controlled lifetime extension in the present C++ language.

```
namespace std
{
    template< typename CharT >
    class basic_string_view
    {
```

```
        public:
            template< typename U >
            basic_string_view( const U &u, std::string &&storage= {} )
            {
                storage= u;
                *this= basic_string_view( u );
            }
    };
}
```

In the above technique, the lifetime of `storage`, if relying upon the default argument initialization, is until the end of the line which invoked the constructor. This is because the temporary object `std::string` created at the calling site is going to live that long, and the `storage` name is a reference to that object. Because of this, the lifetime of the string that we assign to `storage` is equivalent to the necessary lifetime. The lambda in the constructor invocation captures the `storage` variable which permits us to gain access to modify the storage for which we wish to extend the lifetime.

Proposed Implementation

```
namespace std
{
    template< typename T, typename = void >
    struct is_temp_string_convertible : std::false_type {};

    template< typename T >
    struct is_temp_string_convertible< T, std::void_t< decltype( std::declval< T >().operator std::string () ) > >
            : std::true_type {};

    template< typename T >
    struct is_temp_string_convertible< T, std::void_t< decltype( std::as_const( std::declval< T >() ).operator std::string () ) > >
            : std::true_type {};

    template< typename CharT >
    class basic_string_view
    {
            // ...
        public:
            class prevent_abuse { friend basic_string_view; abuse()= default; };

            template
            <
                typename AlienType
                typename= std::enable_if
                <
                    std::is_convertible< AlienType, const char * >::value
                    || std::is_convertible< AlienType, std::string >::value
                    || std::is_convertible< AlienType, const std::string & >::value,
                    void
                >::type,
            >
            basic_string_view( const AlienType &a, prevent_abuse= {}, std::string &&lifetime_extension= "" )
                    : basic_string_view
                        (
                            [&a, &lifetime_extension] () -> basic_string_view
                            {
                                if constexpr( is_temp_string_convertible< AlienType >::value )
                                {
                                    // The assignment to `lifetime_extension` preserves the storage
                                    // of the string we will reference.
                                    lifetime_extension= a.operator std::string();
                                    return lifetime_extension;
                                }
                                else if constexpr( std::is_convertible< AlienType, const char * >::value )
                                {
                                    return static_cast< const char * >( a );
                                }
                                else
                                {
                                    return static_cast< const std::string & >( a );
                                }
                            }()
                        )
            {}
            // ...
    };
}
```

The above multiply-constrained template constructor is able to work as a conversion constructor only when the `AlienType` has a conversion to at least one of the "classical" string types. If-constexpr is used to select among 3 possible implementations, where each variation is used to select the correct conversion target type. It is important to use a `const char *` conversion as preferred over a `const std::string &` conversion, since the `const std::string &` would wind up creating a temporary, when used as a conversion target for a `const char *` targetting conversion. The first variation in the `if constexpr` uses the earlier discussed lifetime extension trick to preserve the storage for the ephemeral string returned by the user-defined conversion for the `AlienType`.

The purpose of the `prevent_abuse` structure as a parameter is to guard this new augmented constructor against direct call by a user. The `prevent_abuse` ctor is private and thus not callable outside of the class; however, C++ rules dictate that the protection and lookup rules for a default parameter are to use the permissions of the context of the definition of the defaulted parameter. What this means is that the ctor is permitted to construct `prevent_abuse` as part of its default arguments, but a user cannot directly call that constructor on his or her own. Thus this constructor is ONLY a conversion operation, and it is only accessible if the user has defined a "classical string" conversion.

Avenues for Further Research

There are a few techniques explored in this paper, which may have general applications in the future, and they should be researched more thoroughly:

- Disabling direct user access to default arguments in constructors through privileged tag classes
- Synthetic lifetime extension, through defaulted constructor rvalue-reference parameters
- Generalization of the `std::optional< T >::optional( U &&u )` "invisible" conversion forwarding constructor technique, if P0892R0 ("Constexpr(bool)") is not approved.

Conclusion

We have presented a functional constructor for `std::string_view` implemented in the C++17 language which solves the stated problem. Our implementation experience is presently limited to a standalone limited reimplementation of the C++17 `string_view` type in a non-`std` namespace. Several testing types are included in our demonstration, which exercise every `if-constexpr` branch. We propose that this constructor be considered for inclusion as part of `std::basic_string_view` in the next C++ standard.